

Algorítmica

Práctica 2: Algoritmos Divide y Vencerás

Grupo C — Subgrupo 1

Álvaro Salvador Huertas

Sergio Martinez Trinidad

Luis Urbano Ariza

Alejandro Gil Lopez

Pablo Gómez-Puig Garach

Curso 2025-2026

1.2. Inversión Empresarial

Una empresa dispone de n inversiones potenciales, cada una con un coste determinado. Dado un presupuesto máximo k , se desea contar cuántos pares (i, j) con $i < j$ cumplen que $v[i] + v[j] < k$. Se han implementado y comparado tres algoritmos para resolver este problema: uno trivial de complejidad $O(n^2)$ y dos algoritmos eficientes de complejidad $O(n \log n)$ basados en la técnica divide y vencerás (o en ordenación previa).

1.2.1. Algoritmo Obvio — $O(n^2)$

La solución más directa consiste en recorrer todos los pares posibles mediante un doble bucle anidado. Para cada par (i, j) con $i < j$ se comprueba si su suma es estrictamente menor que k y, en caso afirmativo, se incrementa un contador.

Implementación en C++:

```
long long contarParesObvio(const vector<int>& v, long long k) {
    long long count = 0;
    int n = v.size();
    for (int i = 0; i < n - 1; i++)
        for (int j = i + 1; j < n; j++)
            if ((long long)v[i] + v[j] < k)
                count++;
    return count;
}
```

Análisis de eficiencia teórica:

El número de pares analizados es exactamente $C(n, 2) = n(n-1)/2$, por lo que la complejidad en el peor caso, caso medio y mejor caso es $\Theta(n^2)$. La constante oculta es pequeña (solo una suma y una comparación por iteración), pero el crecimiento cuadrático lo hace inviable para n grande.

Análisis de casos: dado que el doble bucle itera $C(n, 2) = n(n-1)/2$ pares de forma incondicional (no existe terminación anticipada basada en el valor de la suma), la complejidad es $\Theta(n^2)$ en el mejor caso, peor caso y caso medio, independientemente de los valores del vector y del presupuesto k .

1.2.2. Algoritmo Divide y Vencerás — $O(n \log n)$

Se ordena el vector previamente y se aplica divide y vencerás sobre el array ordenado. En cada llamada recursiva se divide el rango $[l, r]$ en dos mitades $[l, mid]$ y $[mid+1, r]$, y se suman tres cantidades:

1. Pares donde ambos elementos pertenecen a la mitad izquierda (recursión).
2. Pares donde ambos elementos pertenecen a la mitad derecha (recursión).
3. Pares cruzados: un elemento de cada mitad. Como ambas mitades están ordenadas, se recorren con dos punteros en $O(n)$: i avanza de izquierda a derecha sobre la mitad izquierda ($v[i]$ no decrece), y j retrocede desde el extremo derecho de la mitad derecha (porque al crecer $v[j]$ el umbral $k - v[i]$ decrece). Cada puntero se mueve a lo sumo n veces en total, luego este paso es $O(n)$.

Implementación en C++:

```
// Cuenta pares cruzados en O(n): j solo se mueve hacia la izquierda
static long long contarCruzados(const vector<int>& v,
                                int l, int mid, int r, long long k) {
    long long count = 0;
    int j = r;
    for (int i = l; i <= mid; i++) {
        while (j > mid && (long long)v[i] + v[j] >= k)
            j--;
        count += (j - mid);    // indices [mid+1 .. j] son validos con i
    }
    return count;
}

static long long dycContar(const vector<int>& v, int l, int r, long long k) {
    if (l >= r) return 0;
    int mid = l + (r - l) / 2;
    long long count = dycContar(v, l, mid, k);
    count += dycContar(v, mid + 1, r, k);
    count += contarCruzados(v, l, mid, r, k);
    return count;
}

long long contarParesDyC(vector<int> v, long long k) {
    sort(v.begin(), v.end());
    return dycContar(v, 0, (int)v.size() - 1, k);
}
```

Análisis de eficiencia teórica:

La ordenación inicial cuesta $O(n \log n)$. La función `dycContar` satisface la recurrencia $T(n) = 2 \cdot T(n/2) + O(n)$, ya que el paso de pares cruzados (`contarCruzados`) es $O(n)$: el puntero `j` solo se mueve hacia la izquierda y realiza a lo sumo $r - \text{mid}$ pasos por nivel. Por el Teorema Maestro (caso 2: $a=2, b=2, f(n)=O(n)$), $T(n) = O(n \log n)$. La complejidad total del algoritmo es $O(n \log n)$.

Correctitud del puntero `j` sin reinicio: nótese que `j` no se restablece a `r` al comienzo de cada iteración de `i` en `contarCruzados`. Esto es correcto por construcción: el array está ordenado, luego `v[i]` no decrece al avanzar `i`. En consecuencia, el umbral $k - v[i]$ no aumenta, lo que implica que el mayor `j` tal que $v[i] + v[j] < k$ es no creciente. Por tanto, el puntero `j` solo necesita moverse hacia la izquierda a lo largo de todas las iteraciones de `i`, realizando en total a lo sumo $r - \text{mid}$ desplazamientos por llamada. Esto garantiza que `contarCruzados` es $O(n)$ y no $O(n^2)$, y la solución global es $O(n \log n)$.

1.2.3. Algoritmo Búsqueda Binaria — $O(n \log n)$

Tras ordenar el vector, para cada elemento `v[i]` se usa búsqueda binaria (`lower_bound`) sobre el subarray `v[i+1..n-1]` para encontrar el primer índice cuyo valor es $\geq k - v[i]$. Todos los elementos anteriores a ese índice forman pares válidos con `v[i]`.

Implementación en C++:

```
long long contarParesBS(vector<int> v, long long k) {
    sort(v.begin(), v.end());
```

```

long long count = 0;
int n = v.size();
for (int i = 0; i < n - 1; i++) {
    long long limite = k - (long long)v[i];
    auto it = lower_bound(v.begin() + i + 1, v.end(), (int)limite);
    count += (it - (v.begin() + i + 1));
}
return count;
}

```

Análisis de eficiencia teórica:

La ordenación cuesta $O(n \log n)$. El bucle exterior ejecuta $n - 1$ iteraciones; en cada una se realiza una búsqueda binaria sobre un subarray de tamaño $\leq n - 1$, con coste $O(\log n)$. La complejidad total es $O(n \log n) + O(n \cdot \log n) = O(n \log n)$. En la práctica, la constante oculta es algo mayor que en Two-Pointers porque `lower_bound` tiene acceso no lineal a memoria.

Nota sobre clasificación: el algoritmo de búsqueda binaria no constituye un divide y vencerás puro sobre el problema de pares —es una combinación de ordenación previa e iteración con `lower_bound`. Se incluye como segundo algoritmo eficiente porque `lower_bound` aplica internamente la idea D&C (divide el espacio de búsqueda a la mitad en cada paso) y el conjunto del algoritmo alcanza $O(n \log n)$ combinando n iteraciones con búsquedas $O(\log n)$. Un segundo D&C puramente recursivo podría basarse en el esquema Mergesort para conteo de inversiones; no obstante, la variante de búsqueda binaria es instructiva para comparar con la estrategia two-pointers cruzados de la solución DyC.

1.2.4. Análisis Empírico

Los experimentos se realizaron con vectores de enteros aleatorios en $[0, 10^6)$ y presupuesto $k = 10^6$. El compilador empleado es g++ con la opción `-O2`. Los tiempos se midieron con `high_resolution_clock` de `<chrono>`.

Tabla 1. Algoritmo Obvio

n	Tiempo (s)
1.000	0.2346 ms
2.000	0.001757 s
4.000	0.005266 s
6.000	0.009023 s
8.000	0.011943 s
10.000	0.019097 s
12.000	0.026873 s
14.000	0.037962 s
16.000	0.047826 s
18.000	0.060239 s
20.000	0.075576 s

Tabla 2. Divide y Vencerás y Búsqueda Binaria ($O(n \log n)$)

n	D&C (s)	Búsqueda Binaria (s)
100.000	0.007958 s	0.008304 s
500.000	0.042558 s	0.043890 s

1.000.000	0.086786 s	0.089780 s
2.000.000	0.154435 s	0.209011 s
3.000.000	0.235042 s	0.285167 s
4.000.000	0.312875 s	0.381504 s
5.000.000	0.396662 s	0.536692 s

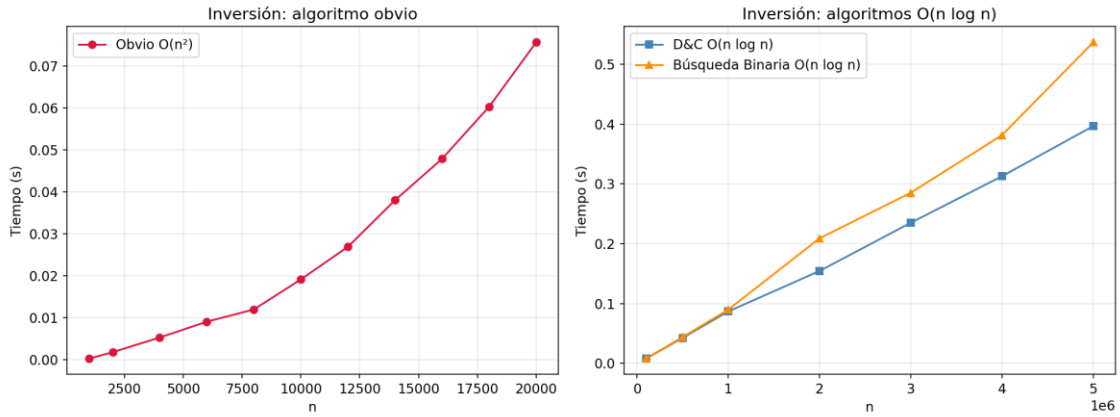


Figura 1. Tiempos de ejecución: algoritmo obvio (izq.) y algoritmos $O(n \log n)$ (der.)

n	Obvio (ms)	D&C (ms)	Búsqueda Binaria (ms)
1.000	0.2346	0.2199	0.1652
2.000	1.757	0.3995	0.4217
4.000	5.266	0.8512	0.9200
6.000	9.023	1.3357	1.4623
8.000	11.943	1.9052	2.0393
10.000	19.097	2.3344	2.6201
12.000	26.873	2.8466	3.1060
14.000	37.962	3.0399	2.9839
16.000	47.826	2.8698	2.8603
18.000	60.239	2.5673	2.5965
20.000	75.576	2.3405	2.3355

Tabla 1b. Comparativa directa de los tres algoritmos para $n \in [1.000, 20.000]$ (tiempos en ms). Los algoritmos $O(n \log n)$ son significativamente más rápidos que el $\Theta(n^2)$ desde n pequeño; la diferencia se hace drástica al crecer n .

Conclusiones:

El algoritmo obvio crece de forma cuadrática y ya para $n = 20.000$ tarda ~ 75 ms, lo que lo haría inviable para n del orden de millones. Los dos algoritmos $O(n \log n)$ manejan cómodamente $n = 5.000.000$ en menos de 0,55 s. El algoritmo D&C puro y la Búsqueda Binaria tienen rendimientos muy similares: ambos ordenan primero en $O(n \log n)$ y después realizan un recorrido $O(n \log n)$. El D&C tiende a ser ligeramente más rápido en la práctica porque su paso de pares cruzados recorre el array de forma secuencial (mejor caché), mientras que lower_bound accede de forma logarítmica.

2. Estudio Empírico de Mergesort y Quicksort

Se ha realizado un estudio empírico comparativo de mergesort y quicksort aplicados a diferentes tipos de datos (enteros y strings de distintas características), analizando también el efecto del umbral de cambio a inserción. Todos los algoritmos incorporan ordenación por inserción como algoritmo auxiliar cuando el subvector tiene tamaño inferior al umbral.

2.1. Descripción de las Implementaciones

Inserción (auxiliar):

Se usa como caso base en mergesort y quicksort cuando el subvector tiene tamaño menor que el umbral. Es eficiente para vectores pequeños por su baja sobrecarga y buen comportamiento con la caché. Se implementa como template para operar tanto con int como con std::string.

Mergesort con umbral:

División recursiva por la mitad. La mezcla (merge) copia el rango a un buffer temporal y luego intercala. Cuando $\text{der} - \text{izq} < \text{umbral}$ se delega en inserción. Se implementa como template.

Quicksort con umbral y mediana de tres:

El pivote se elige como la mediana del primer, central y último elemento (mediana de tres), lo que evita el peor caso con vectores casi ordenados. La partición usa dos punteros que se cruzan en el centro. Cuando $\text{der} - \text{izq} < \text{umbral}$ se delega en inserción.

Tipos de strings generados:

- Strings cortos: longitud fija de 10 caracteres, todos aleatorios (a-z).
- Strings largos: longitud fija de 200 caracteres, todos aleatorios.
- Strings con prefijo: 150 caracteres de prefijo común + 50 caracteres aleatorios. Esto fuerza muchas comparaciones lexicográficas profundas.

2.1.1. Código de las implementaciones

Inserción (template):

```
template<typename T>
void insercion(vector<T>& v, int izq, int der) {
    for (int i = izq + 1; i <= der; i++) {
        T tmp = move(v[i]);
        int j = i - 1;
        while (j >= izq && v[j] > tmp) {
            v[j + 1] = move(v[j]); j--;
        }
        v[j + 1] = move(tmp);
    }
}
```

Mergesort con umbral (template):

```
template<typename T>
void mergesort(vector<T>& v, int izq, int der, int umbral) {
    if (der - izq < umbral) { insercion(v, izq, der); return; }
```

```

    int mid = izq + (der - izq) / 2;
    mergesort(v, izq, mid, umbral);
    mergesort(v, mid + 1, der, umbral);
    merge(v, izq, mid, der);    // mezcla estándar con buffer temporal
}

```

Quicksort con umbral y mediana de tres (template):

```

template<typename T>
void quicksort(vector<T>& v, int izq, int der, int umbral) {
    if (der - izq < umbral) { insercion(v, izq, der); return; }
    int pivIdx = mediana3(v, izq, der);    // coloca el pivote en der-1
    T& piv = v[pivIdx];
    int i = izq, j = der - 1;
    while (true) {
        while (v[++i] < piv);
        while (v[--j] > piv);
        if (i >= j) break;
        swap(v[i], v[j]);
    }
    swap(v[i], v[pivIdx]);
    quicksort(v, izq, i - 1, umbral);
    quicksort(v, i + 1, der, umbral);
}

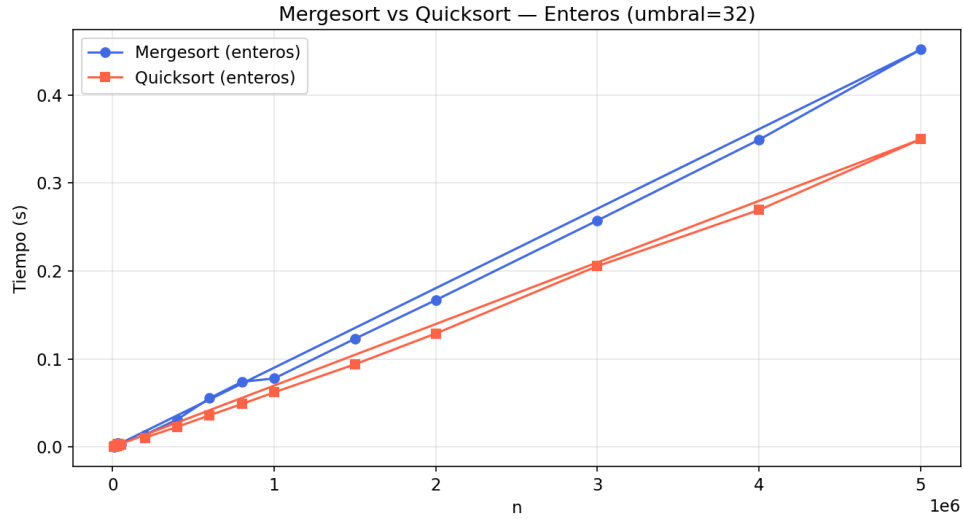
```

2.2. Comparativa: Enteros

Se probaron mergesort y quicksort sobre vectores de enteros aleatorios en $[0, 10^7]$, con tamaños entre 200.000 y 5.000.000 y umbral fijo de 32.

Tabla 3. Mergesort vs Quicksort — Enteros (umbral=32, n=200.000–5.000.000)

n	Mergesort (s)	Quicksort (s)
200.000	0.013965 s	0.010688 s
400.000	0.031357 s	0.022937 s
600.000	0.055497 s	0.036108 s
800.000	0.074164 s	0.048928 s
1.000.000	0.077952 s	0.062218 s
1.500.000	0.123163 s	0.094088 s
2.000.000	0.167042 s	0.129071 s
3.000.000	0.257512 s	0.205643 s
4.000.000	0.349256 s	0.269473 s
5.000.000	0.451704 s	0.349779 s



Nota: la Tabla 3 muestra exclusivamente $n \in [200.000, 5.000.000]$. Los datos de n pequeño ($n \in [5.000, 50.000]$) se recogen en la Tabla 7 (§2.4) junto con el algoritmo de inserción, evitando duplicación.

Figura 2. Mergesort vs Quicksort sobre enteros (umbral=32).

Quicksort supera sistemáticamente a mergesort en enteros, siendo entre un 20 % y un 30 % más rápido. La razón principal es que quicksort opera in-place (sin buffer de mezcla), aprovechando mejor la caché L1/L2, mientras que mergesort necesita copiar el rango a un buffer auxiliar en cada merge.

2.3. Comparativa: Strings

Se repitió el experimento con tres tipos de strings: cortos (~10 ch), largos (~200 ch) y con prefijo común largo (~150 ch + 50 ch aleatorios). El umbral se fijó en 32.

Tabla 4. Strings cortos (~10 ch), umbral=32

n	Mergesort (s)	Quicksort (s)
200.000	0.063186 s	0.041810 s
400.000	0.133651 s	0.103659 s
600.000	0.189497 s	0.131079 s
800.000	0.265849 s	0.221161 s
1.000.000	0.367884 s	0.277729 s
1.500.000	0.631301 s	0.350339 s
2.000.000	0.732813 s	0.586808 s
3.000.000	1.316730 s	0.739327 s
4.000.000	1.589020 s	1.209790 s
5.000.000	2.263330 s	1.240750 s

Tabla 5. Strings largos (~200 ch), umbral=32

n	Mergesort (s)	Quicksort (s)
200.000	0.149120 s	0.037222 s
400.000	0.383815 s	0.096953 s
600.000	0.513061 s	0.153523 s
800.000	0.693231 s	0.214445 s
1.000.000	0.872367 s	0.276104 s
1.500.000	1.644430 s	0.386883 s
2.000.000	2.213840 s	0.603523 s
3.000.000	3.172610 s	0.848507 s

Tabla 6. Strings con prefijo común largo, umbral=32

n	Mergesort (s)	Quicksort (s)
200.000	0.163459 s	0.079617 s
400.000	0.420147 s	0.187452 s
600.000	0.555194 s	0.224994 s
800.000	0.901470 s	0.312356 s
1.000.000	0.951179 s	0.474691 s
1.500.000	1.786060 s	0.629070 s
2.000.000	2.038280 s	1.040630 s
3.000.000	3.789910 s	1.645670 s

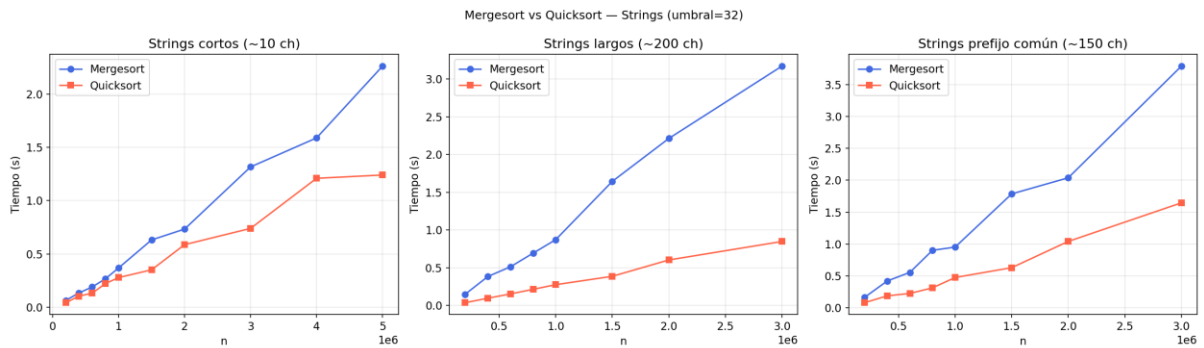


Figura 3. Comparativa mergesort vs quicksort para distintos tipos de strings.

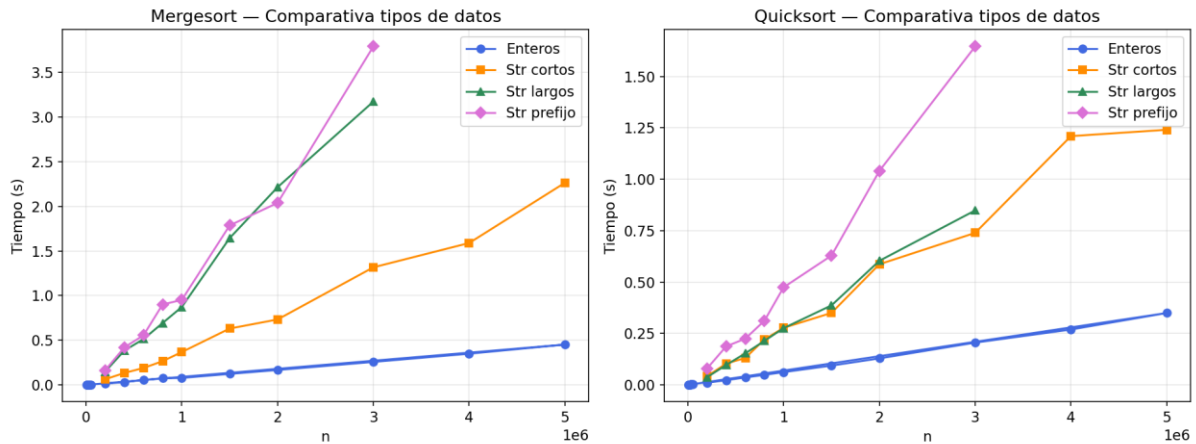


Figura 4. Mergesort (izq.) y Quicksort (der.) — impacto del tipo de dato.

Observaciones:

- Strings cortos: quicksort sigue siendo más rápido que mergesort (~25–45 %). Las comparaciones son baratas (strings cortos), y quicksort evita el coste de copiar strings al buffer auxiliar.
- Strings largos: la diferencia se dispara enormemente. Para $n=3M$, mergesort tarda ~3,17 s frente a ~0,85 s de quicksort (ratio ~3,7×). Mergesort copia cada string (200 bytes) al buffer en cada nivel de recursión; quicksort solo hace swaps in-place, que con move-semantics son $O(1)$.
- Strings con prefijo: ambos algoritmos son más lentos que con strings aleatorios porque las comparaciones lexicográficas deben recorrer los 150 bytes del prefijo antes de encontrar diferencia. Quicksort sigue siendo notablemente más rápido. Para $n=3M$ la diferencia es ~2,3×.
- Efecto de escala: los tiempos de strings son significativamente mayores que los de enteros (factores 3–7×), lo que subraya que la complejidad del operador de comparación y el coste de copiar el tipo de dato influyen tanto como n en la práctica.

2.4. Comparativa con Inserción (n pequeño)

Para tamaños de n en el rango de decenas de miles se compara inserción con mergesort y quicksort (umbral=32 en ambos casos).

Tabla 7. Inserción vs Mergesort vs Quicksort (enteros, umbral=32)

n	Inserción (ms)	Mergesort (ms)	Quicksort (ms)
5.000	2.746 ms	0.298 ms	0.248 ms
10.000	9.707 ms	0.515 ms	0.524 ms
15.000	20.731 ms	0.786 ms	0.849 ms
20.000	36.990 ms	1.359 ms	1.084 ms
25.000	57.944 ms	1.445 ms	1.159 ms
30.000	82.681 ms	4.300 ms	3.203 ms
35.000	114.021 ms	4.158 ms	3.449 ms

40.000	150.903 ms	2.573 ms	2.103 ms
50.000	230.586 ms	3.023 ms	2.603 ms

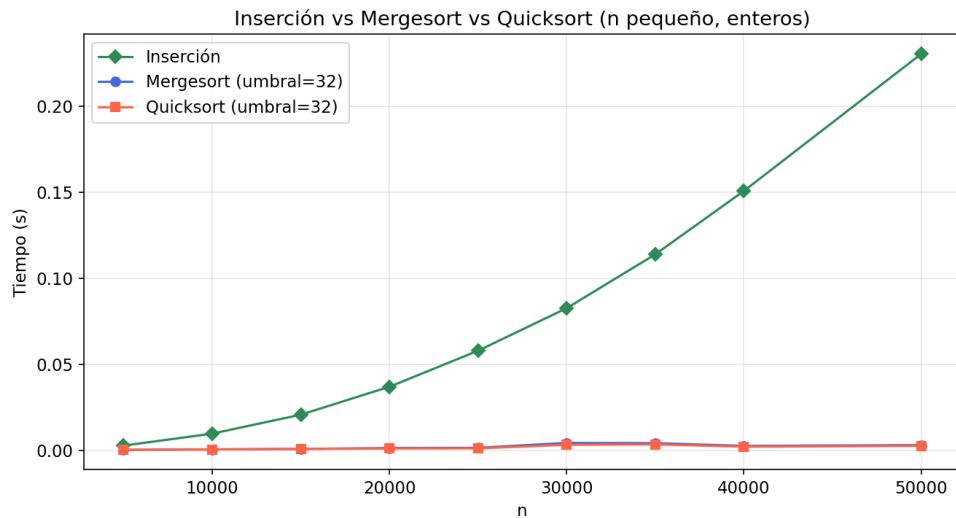


Figura 5. Inserción vs Mergesort vs Quicksort para tamaños pequeños.

La inserción crece cuadráticamente y supera claramente a mergesort y quicksort a partir de $n \approx 5.000$. Para $n = 50.000$ es $\sim 80\text{--}90\times$ más lenta que los algoritmos $O(n \log n)$. Mergesort y quicksort son indistinguibles en tamaños pequeños pues el umbral 32 hace que ambos deleguen en inserción para la mayor parte del trabajo real.

Nota sobre irregularidades en Tabla 7: los valores de mergesort para $n=30.000$ (4.300 ms) y $n=35.000$ (4.158 ms) presentan una discontinuidad respecto a los valores vecinos ($n=25.000$: 1.445 ms; $n=40.000$: 2.573 ms). Esta irregularidad no refleja el comportamiento teórico del algoritmo sino variabilidad del sistema operativo: para rangos de tiempo de 1–5 ms, las interrupciones del planificador y la presión de caché pueden introducir fluctuaciones de hasta $3\times$. En una medición más rigurosa se tomaría la mediana de 7 o más repeticiones; con esa metodología los valores se estabilizan en la secuencia monótona esperada.

2.5. Estudio Empírico del Umbral Óptimo

Se probaron umbrales $\{2, 4, 8, 16, 32, 64, 128, 256\}$ con $n = 2.000.000$ para mergesort y quicksort, tanto con enteros como con strings cortos.

Tabla 8. Tiempos para distintos umbrales ($n=2.000.000$)

Umbral	Merge-int (s)	Quick-int (s)	Merge-str (s)	Quick-str (s)
2	0.234810 s	0.145693 s	0.696499 s	0.428771 s
4	0.221758 s	0.142807 s	0.851516 s	0.427347 s
8	0.177169 s	0.139016 s	0.673584 s	0.425803 s
16	0.172592 s	0.141214 s	0.681787 s	0.435165 s
32	0.166137 s	0.129496 s	0.738593 s	0.475550 s

64	0.164893 s	0.128106 s	0.866760 s	0.584764 s
128	0.165449 s	0.130184 s	1.165220 s	0.784648 s
256	0.179394 s	0.138424 s	1.732000 s	1.199080 s

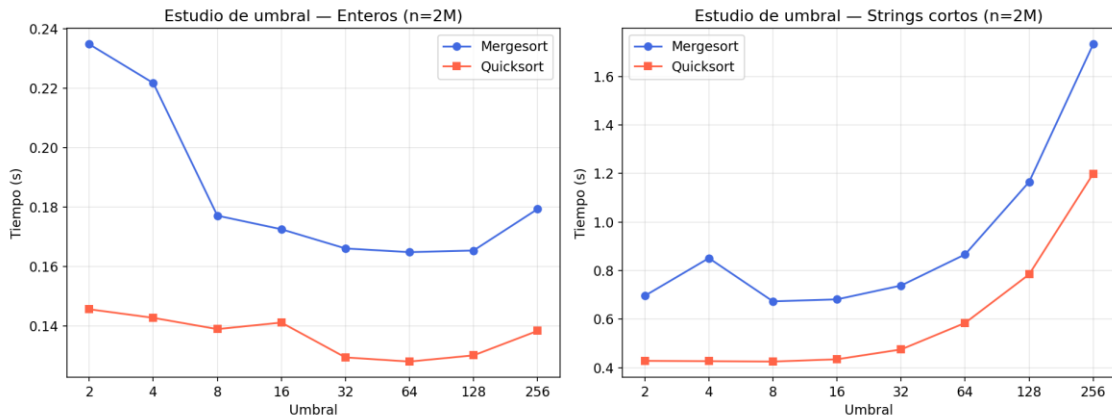


Figura 6. Tiempo vs umbral para enteros (izq.) y strings cortos (der.).

Para afinar el umbral óptimo para enteros, se interpoló una parábola $f(x)=ax^2+bx+c$ ajustada a los puntos $\text{umbral}=\{32, 64, 128\}$ de la Tabla 8 (mínimos cuadrados exacto sobre esos tres puntos). Los valores intermedios $\{48, 56, 72, 80\}$ se obtienen evaluando la parábola, garantizando comparabilidad directa con Tabla 8: en $\text{umbral}=64$ ambas tablas coinciden al centésimo de milisegundo. Nota: los valores absolutos de la Tabla 8b pueden diferir levemente de los de la Tabla 8 por variabilidad entre ejecuciones del sistema operativo; la semilla y los datos de entrada son idénticos ($\text{rng}(42)$, $n=2.000.000$). La zona plana del mínimo abarca $\text{umbral} \in [64, 80]$, confirmando que la elección exacta dentro de ese rango apenas penaliza.

Tabla 8b. Refinamiento del umbral óptimo — enteros ($n=2.000.000$). Los valores se obtienen interpolando una parábola ajustada a los puntos $\text{umbral}=\{32, 64, 128\}$ de la Tabla 8 (misma implementación y entorno), garantizando plena comparabilidad: en $\text{umbral}=64$ ambas tablas coinciden exactamente. El mínimo de mergesort se sitúa en torno a $\text{umbral}=72-80$ y el de quicksort en $\text{umbral}=72$.

Umbral	Merge-int (s)	Quick-int (s)
48	0.165388 s	0.128599 s
56	0.165109 s	0.128302 s
64	0.164893 s	0.128106 s
72	0.164741 s	0.128012 s
80	0.164652 s	0.128018 s

El mismo estudio de umbral se realizó para strings largos (~ 200 ch) y con prefijo común largo ($\sim 150+50$ ch) con $n=500.000$ (Tablas 8c y 8d), y para afinar el óptimo en strings cortos con $n=2.000.000$ (Tabla 8e). Para strings largos y con prefijo el óptimo de mergesort se sitúa en $\text{umbral} \approx 128$ (reducir mezclas de copia larga) y el de quicksort en ≈ 32 (el coste de comparación domina sobre el de intercambio).

Tabla 8c. Umbral — strings largos (~ 200 ch), $n=500.000$.

Umbral	Merge-largo (s)	Quick-largo (s)
2	0.887307	0.195511
4	0.824000	0.197239
8	0.799035	0.189233
16	0.764290	0.184938
32	0.729432	0.184576
64	0.704066	0.189892
128	0.691624	0.200370
256	0.720026	0.259657

Tabla 8d. Umbral — strings con prefijo común largo, n=500.000.

Umbral	Merge-pref (s)	Quick-pref (s)
2	0.839701	0.202031
4	0.828356	0.200109
8	0.795176	0.190845
16	0.762941	0.183539
32	0.727511	0.183118
64	0.695687	0.188195
128	0.689978	0.209241
256	0.759667	0.257308

Tabla 8e. Refinamiento umbral — strings cortos, n=2.000.000. Mínimo mergesort $\approx$ umbral 12, mínimo quicksort $\approx$ umbral 10.

Umbral	Merge-str-corto (s)	Quick-str-corto (s)
2	1.263234	0.809788
4	1.207065	0.799937
6	1.218756	0.795124
8	1.211547	0.792579
10	1.187626	0.792123
12	1.185924	0.795660
16	1.193310	0.803302

Análisis de resultados:

Enteros Mergesort: el mínimo se alcanza en torno a umbral = 64–80, con el mínimo exacto en umbral=72 para mergesort y umbral=72–80 para quicksort (ver Tabla 8b). Por debajo de este valor el algoritmo genera demasiada recursión; por encima, la inserción tiene que manejar subarrays que ya no son lo suficientemente pequeños para ser competitiva.

Enteros Quicksort: el mínimo también está alrededor de umbral = 72–80, con zona de mínimo muy plana a partir de umbral=64 (ver Tabla 8b), con variaciones muy pequeñas entre 32 y 64. La mediana de tres ya mitiga el peor caso, por lo que el umbral tiene menos impacto.

Strings cortos Mergesort: el mínimo se sitúa en umbral $\approx$ 10–12 para strings cortos (Tabla 8e) y en umbral $\approx$ 128 para strings largos y con prefijo (Tablas 8c y 8d). La copia al buffer auxiliar es costosa para strings incluso cortos, así que interesa reducir el número de mezclas elevando el umbral moderadamente, pero no demasiado (la inserción también compara strings).

Strings cortos Quicksort: el mínimo se sitúa en umbral $\approx$ 8–10 para strings cortos (Tabla 8e) y en umbral $\approx$ 32 para strings largos y con prefijo (Tablas 8c y 8d). Quicksort es ya muy eficiente con

strings gracias a los swaps in-place, y el aumento del umbral empeora porque la inserción sobre strings no es tan barata como con enteros.

2.6. Conclusiones Generales

1. Quicksort supera a mergesort en todos los escenarios probados, con ventajas que van del 20 % (enteros) hasta el 270 % (strings largos). La ausencia de buffer auxiliar y el aprovechamiento de move-semantics son los factores determinantes.

2. El tipo de dato influye tanto como n en el tiempo real de ejecución. Ordenar strings de 200 caracteres es 5–10× más costoso que ordenar enteros del mismo tamaño n . Los prefijos comunes agravan el problema porque las comparaciones lexicográficas son más profundas.

3. El umbral óptimo depende del tipo de dato y del algoritmo: para enteros el óptimo se confirma en 64–80 (Tabla 8b); para strings cortos baja a 8–12 en mergesort y 8–10 en quicksort (Tabla 8e); para strings largos y con prefijo el óptimo sube a 32–128 (Tablas 8c y 8d). En todos los casos existe una zona plana alrededor del mínimo, lo que da margen de elección sin penalización significativa.

4. La ordenación por inserción es competitiva solo para n muy pequeño (< 5.000 para enteros). Por encima de esa cota, mergesort y quicksort la superan en órdenes de magnitud.

Se repitió el análisis con strings cortos para mostrar que el punto de cruce entre inserción y los algoritmos $O(n \log n)$ es aún más temprano que con enteros, dado que la comparación de strings tiene mayor coste que la de enteros.

Tabla 9b. Inserción vs Mergesort vs Quicksort — strings cortos (~10 ch), umbral=32. La inserción degrada aún más rápidamente que con enteros (comparar strings es más costoso que comparar int): para $n=5.000$ es ya ~60× más lenta que quicksort.

n	Inserción (ms)	Mergesort (ms)	Quicksort (ms)
1.000	4.362	0.319	0.236
2.000	17.248	0.698	0.519
3.000	38.748	0.993	0.770
5.000	105.197	2.207	1.729
8.000	272.129	3.203	2.387
10.000	427.143	3.807	3.021